PROJECT COMPLETION REPORT (PCR)

AI COMPANY BRAIN — Enterprise GraphRAG Knowledge Platform

Project Name: AI Company Brain
Team Name: CodeSmiths
Project Type: Enterprise AI / GraphRAG / Knowledge Intelligence Platform
Project Duration: July 2026
Primary Domain: Artificial Intelligence, Knowledge Graphs, Retrieval-Augmented Generation (RAG), Enterprise Search

---

1. EXECUTIVE SUMMARY

AI Company Brain is an enterprise-grade GraphRAG (Graph Retrieval-Augmented Generation) knowledge platform designed to solve the problem of fragmented organizational knowledge by combining semantic retrieval, vector databases, knowledge graphs, graph traversal, and large language model reasoning.

Unlike traditional enterprise search systems that perform keyword matching over disconnected documents, AI Company Brain constructs a semantic and relational understanding of organizational knowledge and provides explainable, citation-backed answers.

The project evolved from a multi-team academic collaboration into a fully operational AI knowledge platform consisting of:

- Semantic retrieval
- Vector search
- Knowledge graph reasoning
- Graph expansion
- LLM synthesis
- Citation generation
- Agent orchestration
- Enterprise dashboard
- Interactive graph visualization

The final system successfully demonstrates a complete GraphRAG pipeline using Qdrant, Neo4j, Sentence Transformers, Gemini, FastAPI, React, and Docker.

---

2. PROBLEM STATEMENT

Modern organizations suffer from knowledge fragmentation.

Information exists across:

- Internal documentation
- Incident reports
- PDFs
- Slack discussions
- GitHub repositories
- Knowledge bases
- Notion pages
- Engineering runbooks

Traditional enterprise search systems fail because they:

- Perform keyword matching only
- Cannot understand semantic meaning
- Cannot reason about relationships
- Cannot explain answers
- Cannot cite information sources
- Frequently hallucinate

This results in:

- Poor knowledge discovery
- Lost organizational context
- Slow incident resolution
- Duplicate effort
- Reduced productivity

The objective of AI Company Brain was to create an explainable enterprise intelligence system capable of semantic understanding and graph-based reasoning.

---

3. PROJECT OBJECTIVES

Primary objectives:

- Build an enterprise AI knowledge platform
- Implement semantic retrieval
- Implement knowledge graph reasoning
- Create GraphRAG architecture
- Generate explainable answers
- Support source citations

- Provide interactive visualizations
- Create production-style enterprise UI

Secondary objectives:

- Learn GraphRAG architecture
- Explore vector databases
- Explore graph databases
- Build a complete AI system
- Create a portfolio-quality project

---

4. TEAM STRUCTURE

Team Lead

Aayaan

Responsibilities:

- AI architecture design
- GraphRAG pipeline
- Vector retrieval
- Knowledge graph integration
- Graph expansion
- LLM synthesis
- Citation engine
- Backend integration
- Infrastructure setup
- Docker orchestration
- Final system integration
- Deployment and testing

Backend Ingestion Team

Nikshita

Responsibilities:

- Document ingestion pipeline
- Upload interfaces
- Document processing
- Metadata extraction
- Backend APIs

- File connector integration

Purpose:

The ingestion subsystem acts as the entry point for enterprise knowledge.

Frontend Team

Thanmayee

Responsibilities:

- React frontend
- Dashboard
- Authentication
- User interface
- Navigation
- Graph visualization
- Admin pages
- Design system

Purpose:

Provides the enterprise user experience layer.

Agent Systems Team

Responsibilities:

- Planner Agent
- Verification Agent
- Audit Agent
- Tool Executor
- Workflow Orchestrator
- Confirmation Framework

Purpose:

Provides deterministic agent orchestration infrastructure.

---

5. PROJECT EVOLUTION

Initial Vision

Original plan:

Create an enterprise AI assistant capable of understanding organizational knowledge.

Initial components:

- Search
- Chatbot
- File upload
- Dashboard

Discovery Phase

During implementation we discovered:

Traditional RAG suffers from:

- Poor context
- Hallucinations
- Weak reasoning
- Missing relationships

Decision:

Move from traditional RAG to GraphRAG.

Architecture Pivot

Original:

User ↓ Vector Search ↓ LLM

New:

User ↓ Embeddings ↓ Qdrant ↓ Graph Retrieval ↓ Graph Expansion ↓ LLM ↓ Citation Engine

---

6. COMPLETE SYSTEM ARCHITECTURE

USER ↓ React Frontend ↓ FastAPI Backend ↓ GraphRAG Orchestrator ↓ Sentence Transformers ↓ Qdrant Vector Database ↓ Neo4j Knowledge Graph ↓ Graph Expansion ↓ Gemini 2.5 Flash ↓ Citation Engine ↓ Explainable Answer

---

7. TECHNOLOGY STACK

Frontend

- React
- Vite
- TailwindCSS
- JavaScript
- React Router

Purpose:

Enterprise UI development.

Backend

- Python
- FastAPI
- Pydantic
- Uvicorn

Purpose:

API layer and orchestration.

Artificial Intelligence

- Sentence Transformers
- all-MiniLM-L6-v2
- Gemini 2.5 Flash

Purpose:

Semantic understanding and reasoning.

Vector Database

Qdrant

Purpose:

Stores semantic embeddings.

Responsibilities:

- Vector storage
- Similarity search
- Semantic retrieval
- Dense retrieval

Knowledge Graph

Neo4j

Purpose:

Stores relationships.

Responsibilities:

- Entity storage
- Relationship storage
- Graph traversal
- Multi-hop reasoning

Infrastructure

- Docker
- Docker Desktop
- WSL2

Purpose:

Containerized execution.

Development Tools

- Git
- GitHub
- Antigravity
- VS Code
- PowerShell

---

8. SUBSYSTEMS IMPLEMENTED

1. Embedding Subsystem

Implemented:

- Sentence Transformer provider
- Document pipeline
- Chunk pipeline
- Embedding generation

Model:

all-MiniLM-L6-v2

Vector size:

384 dimensions

Status:

SUCCESSFULLY IMPLEMENTED

---

2. Qdrant Integration

Implemented:

- Collection creation
- Vector insertion
- Semantic search
- Retrieval interface

Status:

SUCCESSFULLY IMPLEMENTED

---

3. Knowledge Graph

Implemented:

- Node creation
- Relationship creation
- Traversal
- Neighborhood discovery

Entities:

- Service
- Incident
- Document
- Thread

Status:

SUCCESSFULLY IMPLEMENTED

---

4. Graph Expansion

Implemented:

- BFS traversal
- Multi-hop expansion
- Context enrichment

Purpose:

Convert retrieval results into relational context.

Status:

SUCCESSFULLY IMPLEMENTED

---

5. GraphRAG

Implemented:

- Retrieval
- Expansion
- Prompt building

- Reasoning
- Synthesis

Status:

SUCCESSFULLY IMPLEMENTED

---

6. Citation Engine

Implemented:

- Source tracking
- Provenance mapping
- Citation generation
- Explainability

Status:

SUCCESSFULLY IMPLEMENTED

---

7. Frontend Dashboard

Implemented:

- Dashboard
- Chat
- Knowledge graph
- Architecture view

Status:

SUCCESSFULLY IMPLEMENTED

---

8. Agent Framework

Implemented:

- Planner
- Verifier
- Auditor
- Tool executor
- Workflow engine

Status:

SUCCESSFULLY IMPLEMENTED

---

9. KNOWLEDGE GRAPH STRUCTURE

Implemented graph:

Payment Service | CAUSED | Incident #1001 | DISCUSSED_IN | Slack Thread | REFERENCES | Redis Cluster

This graph is used for:

- Relationship discovery
- Root cause analysis
- Multi-hop reasoning

---

10. GRAPHRAG PIPELINE

User Query ↓ Embedding Generation ↓ Qdrant Retrieval ↓ Document Selection ↓ Neo4j Expansion ↓ Graph Traversal ↓ Prompt Construction ↓ Gemini Reasoning ↓ Citation Generation ↓ Final Answer

---

11. MAJOR PROBLEMS ENCOUNTERED

Python Version Conflicts

Problem:

Python 3.14 caused dependency issues.

Solution:

Downgraded to Python 3.12.

Docker Installation

Problem:

Docker Desktop failed due to outdated WSL.

Solution:

Updated WSL subsystem.

Qdrant Integration

Problem:

Container conflicts.

Solution:

Managed existing containers.

Neo4j Authentication

Problem:

Unknown credentials.

Solution:

Inspected Docker runtime configuration.

Git Branching

Problem:

Multiple feature branches.

Solution:

Created:

- integration
- demo
- main

merge workflow.

Merge Conflicts

Problem:

agents.md context.md plan.md .gitignore

Solution:

Manual conflict resolution.

Frontend Mock Data

Problem:

Frontend used fake services.

Solution:

Connected frontend to real backend APIs.

Backend Dependencies

Problem:

Full backend required SQLAlchemy and Redis.

Solution:

Built lightweight GraphRAG demo server.

---

12. FEATURES PLANNED BUT DROPPED

Initially planned:

- PostgreSQL storage
- Redis queues
- Celery workers
- Authentication system
- Multi-user support
- Slack API
- Notion API

- GitHub API
- Email connectors
- Real enterprise connectors

Dropped because:

- Demo scope
- Time constraints
- Infrastructure complexity

---

13. FEATURES ADDED LATER

Added:

- GraphRAG
- Neo4j
- Qdrant
- Graph expansion
- Citation engine
- Explainability
- Production frontend
- Interactive graph
- Architecture visualizations

---

14. TESTING PERFORMED

Implemented:

test_qdrant.py

Verified:

- collection creation
- insertion
- retrieval

---

test_neo4j.py

Verified:

- node creation

- relationship creation
- graph traversal

---

test_graph_expansion.py

Verified:

- graph expansion
- BFS traversal

---

test_graph_rag.py

Verified:

- end-to-end GraphRAG

---

test_citations.py

Verified:

- citation generation

---

15. FINAL DEMO

Question:

"What caused the payment outage?"

System:

- Retrieved documents
- Expanded graph
- Reasoned using Gemini
- Generated citations

Output:

Redis Cluster caused the payment outage.

Sources:

- payment_incident.md
- redis_outage.md
- slack_thread.md

---

16. WHAT MAKES THIS PROJECT DIFFERENT

Traditional RAG:

Question ↓ Search ↓ LLM ↓ Answer

Problems:

- Hallucinations
- Weak reasoning
- No relationships

AI Company Brain:

Question ↓ Embeddings ↓ Qdrant ↓ Neo4j ↓ Graph Expansion ↓ Gemini ↓ Citations ↓ Explainable Answer

Advantages:

- Semantic retrieval
- Graph reasoning
- Explainability
- Provenance
- Multi-hop context
- Citation support

---

17. SYSTEM REQUIREMENTS

Minimum:

- Windows 10/11
- 8 GB RAM
- Python 3.12
- Node.js 20+
- Docker Desktop

- Git

Recommended:

- 16 GB RAM
- SSD
- WSL2
- Quad-core CPU

---

18. SOFTWARE REQUIREMENTS

Install:

- Python 3.12
- Node.js
- npm
- Docker Desktop
- Git
- WSL2

---

19. STARTING THE PROJECT

Step 1

Start Docker Desktop.

Step 2

Verify containers:

```
docker ps
```

Expected:

- qdrant
- neo4j

Step 3

If stopped:

docker start qdrant

docker start neo4j

Step 4

Start backend:

python demo_server.py

Backend:

http://localhost:8000

Step 5

Verify backend:

http://localhost:8000/api/stats

Step 6

Start frontend:

npm install

npm run dev

Frontend:

http://localhost:5173

Step 7

Login:

admin@codesmiths.ai

Password:

123456789

---

20. SKILLS DEMONSTRATED

- Artificial Intelligence
- GraphRAG
- Retrieval-Augmented Generation
- Knowledge Graphs
- Semantic Search
- Vector Databases
- Neo4j
- Qdrant
- FastAPI
- React
- Docker
- Python
- JavaScript
- LLM Engineering
- Prompt Engineering
- System Design
- API Design
- Graph Traversal
- Explainable AI
- Information Retrieval
- Enterprise Search
- Git
- Team Collaboration

---

21. FUTURE IMPROVEMENTS

- Multi-user support
- RBAC
- Slack connector
- Notion connector
- GitHub connector
- Email connector
- Production deployment
- Observability
- Agentic workflows

- Hybrid retrieval
- Fine-tuned embeddings
- Streaming responses
- Real-time graph updates

---

22. CONCLUSION

AI Company Brain successfully demonstrates that combining semantic retrieval, vector databases, graph databases, and LLM reasoning can produce explainable enterprise intelligence systems.

The project evolved far beyond its original scope and resulted in a fully operational GraphRAG platform featuring:

- Vector retrieval
- Knowledge graphs
- Graph expansion
- LLM reasoning
- Citation generation
- Explainable AI
- Interactive visualization
- Enterprise-grade user experience

The final system represents a complete implementation of a modern GraphRAG architecture and serves as both a functional enterprise knowledge platform and a portfolio-level demonstration of full-stack AI systems engineering.